

Another Candies

Day 5 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“divide the candies into N exactly equal heaps”*
- ▶ *“if the candies can be distributed equally”*

Programming interpretation:

- ▶ The individual redistribution does not matter; only the total candy count matters.
- ▶ Equal heaps are possible exactly when `total_candies` is divisible by `n`.
- ▶ The blank line before each case must be consumed.

Textbook connection: CP4 5.3.9 Modular Arithmetic

Core technique: Divisibility by modulo

- ▶ Each child owns some number of candies.
- ▶ Candies can be redistributed freely.
- ▶ Everyone can receive the same amount exactly when the total is divisible by the number of children.

- ▶ The distribution details do not matter; only the sum matters.
- ▶ Equal sharing means $\text{total} \bmod n$ equals zero.
- ▶ Modulo arithmetic avoids needing to simulate movement.

- ▶ Read the number of test cases.
- ▶ Skip the blank line before each case.
- ▶ Read n and sum the n candy counts.
- ▶ If $\text{total} \% n == 0$, print YES; otherwise print NO.

- ▶ A total of 12 candies among 4 children works because $12 \% 4 = 0$.
- ▶ A total of 13 among 4 does not work because there is a remainder.
- ▶ Large candy counts are fine in Python; other languages may take modulo while summing.

- ▶ If total is divisible by n , each child can receive total / n candies.
- ▶ If total is not divisible by n , no integer equal share exists.
- ▶ The algorithm checks exactly this necessary and sufficient condition.

Complexity: $O(N)$ time per test case and $O(1)$ memory.

- ▶ Handle the blank line in the input format.
- ▶ No greedy or simulation step is required.

Source lines: `anothercandies.py:37-50`

```
import sys

def main():
    input = sys.stdin.buffer.readline

    test_cases = int(input())

    for _ in range(test_cases):

        input()

        n = int(input())
```

What this chunk does

- ▶ The first input line gives the number of test cases.
- ▶ Kattis includes a blank line before each case, so `input()` consumes it.
- ▶ `n` is the number of children and also the divisor for the final check.

Source lines: `anothercandies.py:52-67`

```
total_candies = 0

for _ in range(n):
    candies = int(input())
    total_candies += candies

if total_candies % n == 0:
    print("YES")
else:
    print("NO")

main()
```

What this chunk does

- ▶ The loop adds every child's candy count to one total.
- ▶ `total_candies % n == 0` is the exact equal-distribution test.
- ▶ YES and NO are printed in the required uppercase format.

Solution file: `practice_contests/day_5/anothercandies.py`

Problem: <https://open.kattis.com/problems/anothercandies>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.